

МИНИСТЕРСТВО ПРОСВЕЩЕНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ
им. А.И. ГЕРЦЕНА»

Направление подготовки

09.03.01 – Информатика и вычислительная техника

Профиль «Технологии разработки программного обеспечения»

Лабораторная работа

«Интерполяция»

Работу выполнили студенты 2 курса 1.1 группы:

Шульга Евгений

Зубов Михаил

Содержание

СТР 4 – 7: Интерполяционный многочлен Лагранжа

СТР 8 – 9: Интерполяционная формула Ньютона

СТР 10 – 14: Интерполяция. Полином Стирлинга; Итоги

СТР 15: Список литературы

1. Интерполяционный многочлен Лагранжа

Цель лабораторной работы:

Разработать решение задач по интерполированию функций методом Лагранжа используя методы языка программирования Python

Оборудование:

- Pycharm 2021_3
- Python 3.12 (на старых версиях python может не работать программа)
- Библиотеки для ЯП python: NumPy, Math (необходимо установить для корректной работы программы)

Постановка задачи:

Изучить метод интерполяции функции «Интерполяционный многочлен Лагранжа» реализовать решение используя доступные средства языка программирования Python

Ход работы:

Определение метода:

Пусть x_i ($i = 0, 1, 2, \dots, n$) – произвольные узлы, а $y_i = f(x_i)$ – значения функции $f(x)$. Многочленом степени n , принимающим в точках x_i значения y_i , является *интерполяционный многочлен Лагранжа*.

$$L_n(x) = \sum_{i=0}^n \frac{(x-x_0)(x-x_1)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n)}{(x_i-x_0)(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)} y_i \quad (1)$$

Выражения

$$L_i^{(n)}(x) = \frac{(x-x_0)(x-x_1)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n)}{(x_i-x_0)(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)}$$

называются коэффициентами Лагранжа.

Можно использовать обозначения для удобства записи:

$$\prod_{n+1} = (x-x_0)(x-x_1)\dots(x-x_{i-1})(x-x_{i+1})\dots(x-x_n)$$

$$D_i = (x_i-x_0)(x_i-x_1)\dots(x_i-x_{i-1})(x_i-x_{i+1})\dots(x_i-x_n)$$

С учетом введенных обозначений имеем:

$$f(x) \approx \prod_{n+1} \cdot \sum_{i=0}^n \frac{y_i}{D_i}$$

Нами был использована доступная для некоммерческого использования среда программирования языка Python PYCHARM версии 2021.3 для реализации решения поставленных задач с использованием интерполяционного метода Лагранжа.

Итоговый код программы представлен ниже:

```
import math
import numpy as np

def compute_t(x, x0, h):
    return (x - x0) / h

def compute_Ci(n, i):
    return (-1) ** (n - i) * math.factorial(i) * math.factorial(n - i)

def lagrange_interpolation1(x, x_list, y_list):
    n = len(x_list)
    result = 0.0
    for i in range(n):
        term = y_list[i]
        for j in range(n):
            if j != i:
                term *= (x - x_list[j]) / (x_list[i] - x_list[j])
        result += term
    return result

def lagr14(nodes, values, x):
    n = len(nodes)
    result = 0
    for i in range(n):
        term = values[i]
        for j in range(n):
            if j != i:
                term *= (x - nodes[j]) / (nodes[i] - nodes[j])
        result += term
    return result

# 1.1-----

# Заданные значения из таблицы (неравноотстоящие узлы)
x_list1 = [0.05, 0.10, 0.17, 0.25, 0.30, 0.36]
y_list1 = [0.050042, 0.100335, 0.171657, 0.255342, 0.309336, 0.376403]

# Заданные значения из таблицы (равноотстоящие узлы)
x_list2 = [0.101, 0.106, 0.111, 0.116, 0.121, 0.126]
y_list2 = [1.26183, 1.27644, 1.29122, 1.30617, 1.32130, 1.32660]

# Точки, в которых нужно вычислить значение функции
x_point1 = 0.263
x_point2 = 0.1157

# Вычисляем значения функции в заданных точках
```

```

result1 = lagrange_interpolation1(x_point1, x_list1, y_list1)
result2 = lagrange_interpolation1(x_point2, x_list2, y_list2)

print("\n (1.1) Приближенное значение функции f(x) при x =", x_point1, ":", result1)
print("\n (1.1) Приближенное значение функции f(x) при x =", x_point2, ":", result2)

```

1.2-----

```

x_list = [0.101, 0.106, 0.111, 0.116, 0.121, 0.126]
y_list = [1.26183, 1.27644, 1.29122, 1.30617, 1.32130, 1.32660]
x = 0.1157
n = len(x_list)
h = x_list[1] - x_list[0]

# Вычисляем значение t
t = compute_t(x, x_list[0], h)

# Вычисляем значения  $y_i / ((t - i) * C_i)$  для каждой строки таблицы
values = []
for i in range(n):
    ti = t - i
    Ci = compute_Ci(n - 1, i)
    value = y_list[i] / (ti * Ci)
    print(f"{i} | {x_list[i]} | {y_list[i]} | {ti} | {Ci} | {ti * Ci} | {value}")
    values.append(value)

# Вычисляем значение функции y(x)
result = sum(values)
print("\n (1.2) Значение функции y(x) при x =", x, ":", result)

```

1.3 -----

```

x_nodes = [0, 1/6, 1/2]
y_nodes = [np.sin(np.pi * x) for x in x_nodes]

# Значения x, для которых нужно вычислить полином Лагранжа
x_values = [1/4, 1/3]

# Вычисляем значения полинома Лагранжа для заданных x
for x in x_values:
    y_interpolated = lagrange_interpolation1(x, x_nodes, y_nodes)
    print(f"\n (1.3) Значение интерполяционного полинома Лагранжа для x={x}: {y_interpolated}")

```

1.4 -----

```

# Узлы интерполяции
nodes = [-1, 0, 1]

# Значения функции в узлах
values = [node**2 for node in nodes]

# Точка, в которой нужно вычислить значение интерполяционного полинома
x = 0.5

```

```
# Вычисляем значение интерполяционного полинома Лагранжа в точке x
result = lagr14(nodes, values, x)
print("\n (1.4) Значение интерполяционного полинома в точке", x, ":", result)
```

На изображении (1) представлен результат работы программы:

```
(1.1) Приближенное значение функции f(x) при x = 0.263 : 0.2692364102817679

(1.1) Приближенное значение функции f(x) при x = 0.1157 : 1.3052395713841944
0 | 0.101 | 1.26183 | 2.9400000000000035 | -120 | -352.80000000000004 | -0.0035766156462584994
1 | 0.106 | 1.27644 | 1.9400000000000035 | 24 | 46.56000000000009 | 0.027414948453608198
2 | 0.111 | 1.29122 | 0.9400000000000035 | -12 | -11.280000000000042 | -0.11446985815602795
3 | 0.116 | 1.30617 | -0.0599999999999965 | 12 | -0.719999999999958 | -1.814125000000106
4 | 0.121 | 1.3213 | -1.0599999999999965 | -24 | 25.43999999999916 | 0.05193789308176117
5 | 0.126 | 1.3266 | -2.0599999999999965 | 120 | -247.1999999999996 | -0.005366504854368941

(1.2) Значение функции y(x) при x = 0.1157 : -1.858185137121392

(1.3) Значение интерполяционного полинома Лагранжа для x=0.25: 0.6874999999999999

(1.3) Значение интерполяционного полинома Лагранжа для x=0.3333333333333333: 0.8333333333333333

(1.4) Значение интерполяционного полинома в точке 0.5 : 0.25
```

Изображение 1.

Микро-вывод:

наша группа реализовала метод интерполяции Лагранжа на языке программирования Python, программа выводит результат работы функции, указывая номер задачи. В лабораторной работе имеют место погрешности при вычислении, не выходящие за рамки допустимых данным методом.

2. Интерполяционная формула Ньютона

Цель лабораторной работы:

Разработать решение задач по интерполированию функций методом Ньютона используя методы языка программирования Python

Оборудование:

- Pycharm 2021_3
- Python 3.12 (на старых версиях python может не работать программа)
- Библиотека для ЯП python: Sympy (необходимо установить для корректной работы программы)

Постановка задачи:

Изучить метод интерполяции функции «Интерполяционная формула Ньютона» реализовать решение используя доступные средства языка программирования Python

Ход работы:

Определение:

Интерполяционный полином Ньютона Интерполяционный полином Ньютона – это полином степени n . Здесь n — это количество узлов интерполяции. При его построении используются разделенные разности.

Интерполяционный полином Ньютона определяется следующим образом:

$$P_n(x) = f(x_0) + \sum_{k=1}^n f(x_0; \dots; x_k) \cdot \prod_{i=0}^{k-1} (x - x_i)$$

Нами был использована доступная для некоммерческого использования среда программирования языка Python PYCHARM версии 2021.3 для реализации решения поставленных задач с использованием интерполяционного метода Ньютона.

Код программы представлен ниже


```

from sympy import symbols, simplify

# Заданные данные (таблица значений)
x_values = [1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2.0]
y_values = [0.9427, 0.8802, 0.903, 0.9340, 0.9523, 0.9661, 0.9763, 0.9838, 0.9891, 0.9928, 0.9953]

# Символьные переменные
x = symbols('x')

# Вычисление разделенных разностей
def divided_differences(x_values, y_values):
    n = len(x_values)
    delta = [[0] * n for _ in range(n)]
    for i in range(n):
        delta[i][0] = y_values[i]
    for j in range(1, n):
        for i in range(n - j):
            delta[i][j] = delta[i + 1][j - 1] - delta[i][j - 1]
    return delta

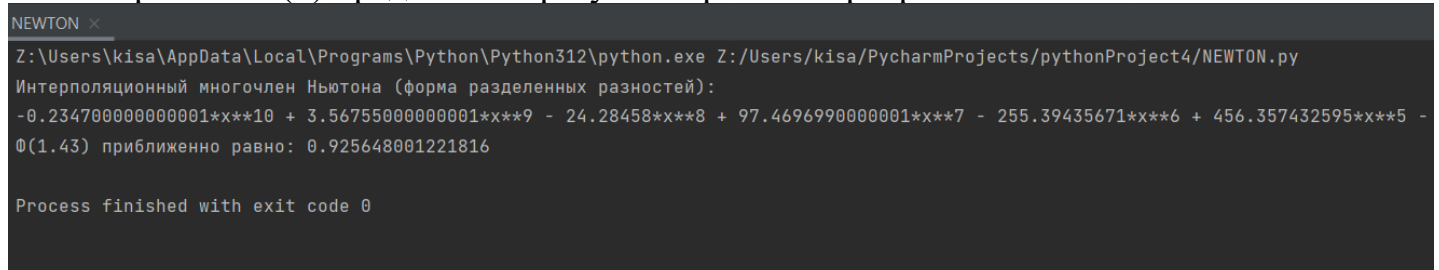
# Построение интерполяционного многочлена
def newton_interpolation_divided_differences(x_values, y_values):
    n = len(x_values)
    delta = divided_differences(x_values, y_values)
    interpolating_polynomial = delta[0][0]
    for i in range(1, n):
        term = delta[0][i]
        for j in range(i):
            term *= (x - x_values[j])
        interpolating_polynomial += term
    return simplify(interpolating_polynomial)

# Вывод интерполяционного многочлена
interpolating_polynomial = newton_interpolation_divided_differences(x_values, y_values)
print("Интерполяционный многочлен Ньютона (форма разделенных разностей):")
print(interpolating_polynomial)

# Подстановка значения x = 1.43
result = interpolating_polynomial.subs(x, 1.43)
print("Φ(1.43) приближенно равно:", result)

```

На изображении (2) представлен результат работы программы:



```

NEWTON ×
Z:\Users\kisa\AppData\Local\Programs\Python\Python312\python.exe Z:/Users/kisa/PycharmProjects/pythonProject4/NEWTON.py
Интерполяционный многочлен Ньютона (форма разделенных разностей):
-0.2347000000000001*x**10 + 3.567550000000001*x**9 - 24.28458*x**8 + 97.4696990000001*x**7 - 255.39435671*x**6 + 456.357432595*x**5 -
562.929668277*x**4 + 472.9693437206*x**3 -
258.645533773832*x**2 + 82.8357629304799*x - 10.768249485248
Φ(1.43) приближенно равно: 0.925648001221816

Process finished with exit code 0

```

Изображение 2.

В изображение в полном объеме не попала форма разделенных разностей, ниже представлен полный её вывод программой:

```

-0.2347000000000001*x**10 + 3.567550000000001*x**9 - 24.28458*x**8 + 97.4696990000001*x**7 -
255.39435671*x**6 + 456.357432595*x**5 - 562.929668277*x**4 + 472.9693437206*x**3 -
258.645533773832*x**2 + 82.8357629304799*x - 10.768249485248

```

Микро-вывод:

наша группа реализовала метод интерполяции Ньютона на языке программирования Python, программа выводит результат работы функции в консоль. В лабораторной работе имеют место погрешности при вычислении, не выходящие за рамки допустимых данным методом.

3. Полином Стирлинга

Цель лабораторной работы:

Разработать решение задач по интерполированию функций методом полинома Стирлинга используя методы языка программирования Python

Оборудование:

- Pycharm 2021_3
- Python 3.12 (на старых версиях python может не работать программа)
- Библиотека для ЯП python: Numpy, Matplotlib (необходимо установить для корректной работы программы)

Постановка задачи:

Изучить метод интерполяции функции «Полином Стирлинга» реализовать решение используя доступные средства языка программирования Python

Ход работы:

Определение:

Интерполяционный полином (многочлен) в общем виде:

$$x(t) = f(t_0) + a_1(t - t_0) + a_2(t - t_0)^2 + a_3(t - t_0)^3 + a_4(t - t_0)^4 + \dots [12]$$

На практике редко требуется применять полиномы больше 4-й степени. Вопрос сводится к вычислению

коэффициентов a_1, a_2, a_3, a_4 . Введем новую независимую переменную – *нормированный аргумент* $\tau = \frac{t - t_0}{h}$:

t	t ₃	t ₂	t ₁	t ₀	t ₁	t ₂	t ₃
τ	-3	-2	-1	0	1	2	3

Тогда $t = t_0 + \tau h$ и [12] можно записать так

$$x(t_0 + \tau h) = f(t_0) + a_1\tau h + a_2\tau^2 h^2 + a_3\tau^3 h^3 + a_4\tau^4 h^4 + \dots$$

$$x(t_0 + \tau h) = A_0 + A_1\tau + A_2\tau^2 + A_3\tau^3 + A_4\tau^4 + \dots [13]$$

Приравняем значения полинома [13] при соответствующих значениях τ и узлов таблицы $x(t_\tau) = x_\tau$, тогда

Уравнения для определения коэффициентов полиномов

t	x(t)	Δ^1	Δ^2	Δ^3	Δ^4
$t_3 = t_0 - 3h$	$A_0 - 3A_1 + 9A_2 - 27A_3 + 81A_4$				
		$\Delta^1_{-5/2} = A_1 - 5A_2 + 19A_3 - 65A_4$			
$t_2 = t_0 - 2h$	$A_0 - 2A_1 + 4A_2 - 8A_3 + 16A_4$		$\Delta^2_{-2} = 2A_2 - 12A_3 + 50A_4$		
		$\Delta^1_{-3/2} = A_1 - 3A_2 + 7A_3 - 15A_4$		$\Delta^3_{-3/2} = 6A_3 - 36A_4$	
$t_1 = t_0 - h$	$A_0 - A_1 + A_2 - A_3 + A_4$		$\Delta^2_{-1} = 2A_2 - 6A_3 + 14A_4$		$\Delta^4_{-1} = 24A_4$
		$\Delta^1_{-1/2} = A_1 - A_2 + A_3 - A_4$		$\Delta^3_{-1/2} = 6A_3 - 12A_4$	
t_0	A_0	$\Delta^1_0 = A_1 + A_3$	$\Delta^2_0 = 2A_2 + 2A_4$	$\Delta^3_0 = 6A_3$	$\Delta^4_0 = 24A_4$
		$\Delta^1_{1/2} = A_1 + A_2 + A_3 + A_4$		$\Delta^3_{1/2} = 6A_3 + 12A_4$	
$t_1 = t_0 + h$	$A_0 + A_1 + A_2 + A_3 + A_4$		$\Delta^2_1 = 2A_2 + 6A_3 + 14A_4$		$\Delta^4_1 = 24A_4$
		$\Delta^1_{3/2} = A_1 + 3A_2 + 7A_3 + 15A_4$		$\Delta^3_{3/2} = 6A_3 + 36A_4$	
$t_2 = t_0 + 2h$	$A_0 + 2A_1 + 4A_2 + 8A_3 + 16A_4$		$\Delta^2_2 = 2A_2 + 12A_3 + 50A_4$		
		$\Delta^1_{5/2} = A_1 + 5A_2 + 19A_3 + 65A_4$			
$t_3 = t_0 + 3h$	$A_0 + 3A_1 + 9A_2 + 27A_3 + 81A_4$				

Каждая запись может рассматриваться как уравнение для определения коэффициентов A_1, A_2, A_3, A_4 . Различные интерполяционные формулы получаются в зависимости от того, какие именно выбираются уравнения для определения этих коэффициентов.

Если использовать только центральные разности, то

$$x(t_0) = A_0,$$

$$\Delta^1_0 = A_1 + A_3,$$

$$\Delta^2_0 = 2A_2 + 2A_4,$$

$$\Delta^3_0 = 6A_3,$$

$$\Delta^4_0 = 24A_4, \text{ отсюда получим } A_4 = \frac{1}{24} \Delta^4_0,$$

$$A_3 = \frac{1}{6} \Delta^3_0,$$

$$A_2 = \frac{1}{2} \Delta^2_0 - \frac{1}{24} \Delta^4_0,$$

$$A_1 = \Delta^1_0 - \frac{1}{6} \Delta^3_0,$$

$$A_0 = x_0.$$

Поэтому интерполяционный многочлен

$$x(t_0 + \tau h) = x_0 + \left[\Delta_0^1 - \frac{1}{6} \Delta_0^3 \right] \tau + \left[\frac{1}{2} \Delta_0^2 - \frac{1}{24} \Delta_0^4 \right] \tau^2 + \frac{1}{6} \Delta_0^3 \tau^3 + \frac{1}{24} \Delta_0^4 \tau^4,$$

раскрывая скобки и располагая слагаемые по разностям Δ :

$$x(t_0 + \tau h) = x_0 + \tau \Delta_0^1 + \frac{1}{2} \tau^2 \Delta_0^2 + \frac{1}{6} (\tau^3 - \tau) \Delta_0^3 + \frac{1}{24} (\tau^4 - \tau^2) \Delta_0^4,$$

$$x(t_0 + \tau h) = S(x_0, \tau) = x_0 + \tau \Delta_0^1 + \frac{\tau^2}{2!} \Delta_0^2 + \frac{\tau(\tau - 1)(\tau + 1)}{3!} \Delta_0^3 + \frac{\tau^2(\tau - 1)(\tau + 1)}{4!} \Delta_0^4$$

Это интерполяционный полином Стирлинга.

Нами был использована доступная для некоммерческого использования среда программирования языка Python PYCHARM версии 2021.3 для реализации решения поставленных задач с использованием полинома стирлинга.

Код программы представлен ниже:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.backends.backend_agg import FigureCanvasAgg
import matplotlib
matplotlib.use('TkAgg')

# Данные значения функции
x_values = np.array([0, 1, 2, 3, 4, 5, 6, 7])
y_values = np.array([0, 2, 5, 10, 15, 20, 22, 24])

# Заданная точка
x_target = 4.3

# Находим ближайшую точку слева и справа от x_target
index_left = np.where(x_values <= x_target)[0][-1]
index_right = np.where(x_values >= x_target)[0][0]

# Определяем узлы для полинома Стирлинга
nodes = [x_values[index_left - 3], x_values[index_left - 2], x_values[index_left - 1],
          x_values[index_right], x_values[index_right + 1], x_values[index_right + 2]]

# Функция для вычисления таблицы разделенных разностей
def divided_differences(x, y):
    n = len(x)
    F = np.zeros((n, n))
    F[:, 0] = y

    for j in range(1, n):
        for i in range(n - j):
            F[i, j] = (F[i + 1, j - 1] - F[i, j - 1]) / (x[i + j] - x[i])
    return F

# Строим таблицу разделенных разностей
table = divided_differences(x_values[index_left - 3:index_right + 3], y_values[index_left - 3:index_right + 3])

# Функция для вычисления полинома Стирлинга
def sterling_polynomial(x, nodes, table, index_left):
    result = 0
    for i in range(len(nodes)):
        term = table[index_left - 3, i]
        for j in range(i):
            term *= (x - nodes[j])
        result += term
    return result
```

```

# Функция для вычисления значения полинома в заданной точке
def sterling_interpolation(x, nodes, table, index_left):
    result = 0
    for i in range(len(nodes)):
        term = table[index_left - 3, i]
        for j in range(i):
            term *= (x - nodes[j])
        result += term
    return result

# Вычисляем значение полинома в заданной точке
result = sterling_polynomial(x_target, nodes, table, 3)

print("Значение функции в точке 4.3:", result)

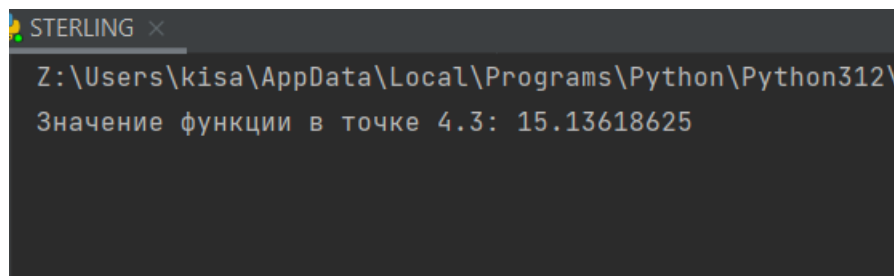
# Значение функции в точке 4.3
y_target = sterling_interpolation(x_target, nodes, table, 3)

# Создаем массив значений для построения графика
x_plot = np.linspace(0, 7, 100)
y_plot = sterling_interpolation(x_plot, nodes, table, 3)

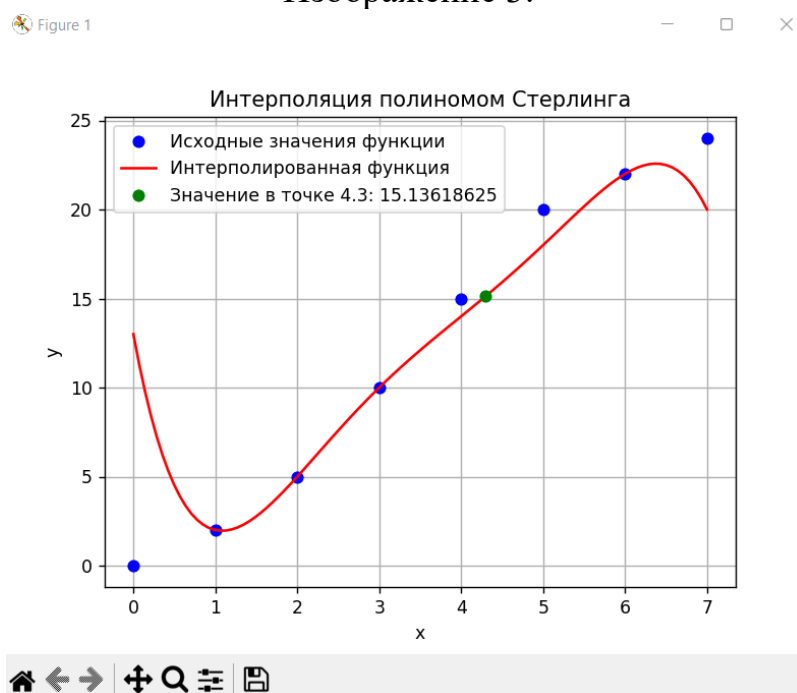
# Строим график функции
plt.plot(x_values, y_values, 'bo', label='Исходные значения функции')
plt.plot(x_plot, y_plot, 'r-', label='Интерполированная функция')
plt.plot(x_target, y_target, 'go', label=f'Значение в точке {x_target}: {y_target}')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Интерполяция полиномом Стерлинга')
plt.legend()
plt.grid(True)
plt.show()

```

На изображении (3) и (4) представлен результат работы программы:



Изображение 3.



Изображение 4.

Микро-вывод:

наша группа реализовала метод интерполяции Стерлинга на языке программирования Python, программа выводит результат работы функции в консоль, также открывается отдельное окно MATPLOTLIB с графиком функции. В ходе реализации задачи были выявлены ошибки, возникающие из-за обновленных методов библиотеки MATPLOTLIB, рекомендуем обновить MATPLOTLIB до версии, актуальной на 06.04.2024 через pip. В лабораторной работе имеют место погрешности при вычислении, не выходящие за рамки допустимых данным методом.

Подведем итоги

Поставленные задачи по реализации методов интерполирования функций для нашей группы были реализованы на языке программирования Python, На основе полученных результатов можно будет сделать выводы о том, какой метод, , более предпочтительный в различных ситуациях. Это может помочь ученым, инженерам и другим специалистам в выборе наиболее подходящего метода интерполяции для своих конкретных задач.

Список литературы

1. национальный университет им. И.И. Мечникова Интерполяционные полиномы Стирлинга, Бесселя, Ньютона / Одесский национальный университет им. И.И. Мечникова [Электронный ресурс] // studfile : [сайт]. — URL: <https://studfile.net/preview/4030410/page:4/> (дата обращения: 06.04.2024).
2. Ижевский государственный технический университет им. М. Т. Калашникова Интерполяционный многочлен лагранжа / Ижевский государственный технический университет им. М. Т. Калашникова [Электронный ресурс] // studfile : [сайт]. — URL: <https://studfile.net/preview/1977996/page:2/> (дата обращения: 06.04.2024).